FILSPAY CHALLENGE · FINAL PITCH

OG

We didn't just listen, we checked.

Ebrahim Alnasser

Bug fixes and root-cause analysis · security hardening and red-team testing · refunds (FP-300) · email OTP · AI assistant guardrails · capacity testing · AI setup

Sayed Hassan Isa

Frontend redesign and theme · "Ask FilsPay" AI assistant · production deployment with HTTPS on Dilmune · second bug pass: 45 more bugs found and fixed · Payout days panel

Live: `og-filspay.139-59-1-82.nip.io` · admin: `admin.og-filspay.139-59-1-82.nip.io` · fork: `github.com/Ananny1/og-fils-pay` @ `990f62e`

ROOT CAUSE

FilsPay's rules lived in comments and docs, not in the code, the database or a test — and false comments plus poisoned AI instructions told everyone not to fix them.

HOW IT SHOWS IN THE CODE	SYMPTOMS IT EXPLAINS
Check, then act, no lock	Link charged twice (BUG-005) · two refunds at once (BUG-034) · 12 OTP guesses past a limit of 5 (BUG-011) · payout notified twice (BUG-008)
Wrong unit or type	CSV 4.75 BD → 475 fils (BUG-014) · `float32` payouts (BUG-021) · `int32` volume (BUG-022)
UTC, not Bahrain	Daily close at 03:00 instead of midnight (BUG-020) · rollups on the wrong day (BUG-055)
Missing scope	Login without OTP (BUG-001) · staff act as shop owner (BUG-010) · refund another shop's sale (BUG-036)
False claim stopped the fix	"Claude says it's redundant" · "already fixed in FP-201" · "money in cents" — 115 traps logged

KEY FINDINGS · RANKED BY IMPACT

55 findings, 24 critical — the top four

#	FINDING	ONE LINE OF EVIDENCE
1	**Login without the OTP** (BUG-001 + session fixation BUG-032)	Password only, then `GET /v1/transactions` with the pre-OTP cookie → **200**. A comment said Claude called the check "redundant".
2	**One payment link charged twice** (BUG-005, incident 2026-05-14)	The 60 s reservation expires during authorize + capture; a second payer re-claims; the first finalize updates 0 rows and still returns 200.
3	**Staff "view as shop" had owner powers** (BUG-010)	Super-admin viewing a shop created a live payment link: **201**, recorded as the shop's own action.
4	**Refunds missing, and their helpers unsafe** (FP-300, BUG-034/035/036)	Route returned 501; two parallel refunds both passed; refundable = authorized (not captured) − refunded; no shop check.

Also: the app didn't build (DEP-001…005), and the repo's own AI files told the AI to keep the bugs (BUG-050).

FIXES AND PROOF

Every fix: failing test first, smallest change, own commit

FIX	WHAT CHANGED	BEFORE	AFTER
BUG-001	Session must be `authenticated`	pre-OTP cookie 200	**401**
BUG-011	Atomic attempt count, `crypto.randomInt`	12 guesses checked	**5**
BUG-010	Every view = read-only support	write 201	**403**
BUG-005	One connection + lock per checkout	double charge	**927 paid = 927 captured** under 3× load + crash
BUG-014	`parseBhdToFils`	4.75 BD → 475 fils	**4750**
FP-300	Refunds end to end (.1–.6)	501	**ledger, batch, webhook, reports tie out**
Tests	Node 22, suites actually run (DEP-005)	426 / 443, 376 never ran	**API 651/651**, reporting 17/17, Go ok

100 bugs found · 100 fixed. First audit: 54 fixed · 1 reclassified (BUG-037). Second pass: 45 more found and fixed (BUG-064–109). Write-up per fix in `docs/bugs/`.

LIVE DEMO · FULL AND PARTIAL REFUND

Pay 10.000 BD → refund 4.000 → refund 6.000 → refused

STEP (DASHBOARD)	EXPECTED
1. Create a 10.000 BD link, pay it at `/pay/<token>`	Captured 10.000 BHD
2. Transaction → **Refund** 4.000 BHD	Status `PARTIALLY REFUNDED`; timeline −4.000
3. Refund the remaining 6.000 BHD	Status `REFUNDED`; refunded 10.000
4. Try one more fil	Refused: 409 "not refundable"
5. Reconciliation	Gap **0.000**, explained
6. Next settlement batch (admin → run)	Net reduced by refunds − fee reversed

Ledger to the fil (MDR 250 bps)

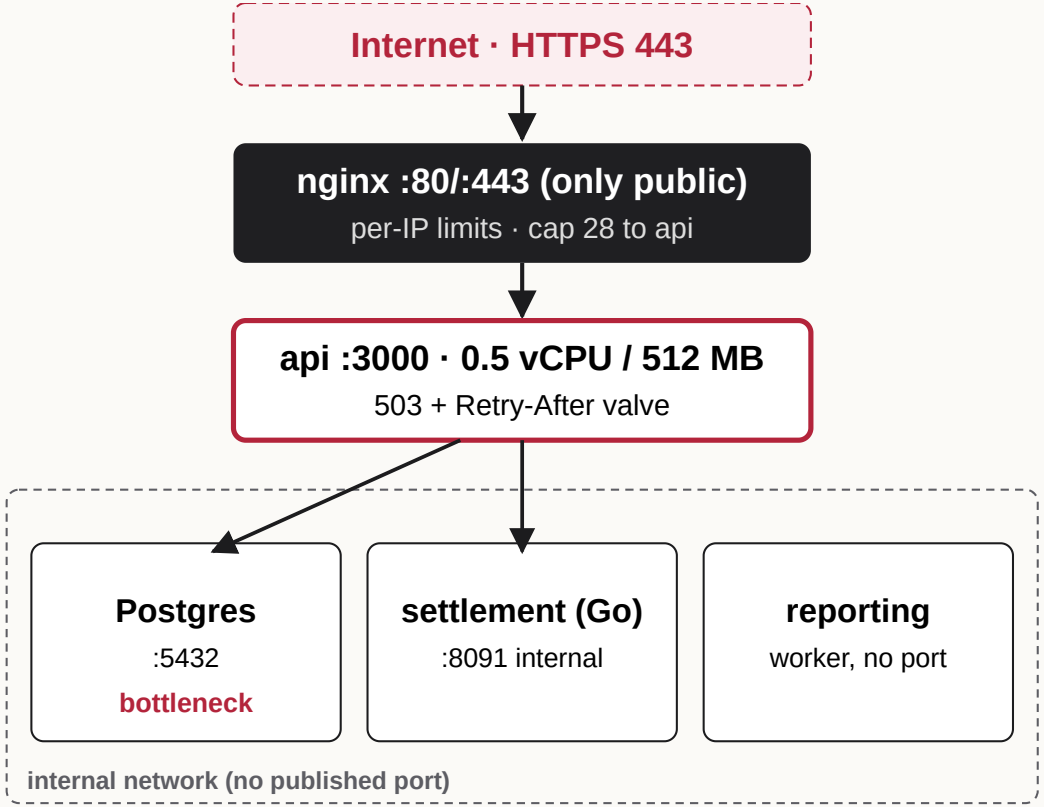
Fee on 10.000	**0.250**
Refund 4.000, fee reversed	0.100
Shop payable after step 2	**5.850**
Refund 6.000, fee reversed	0.150
Shop payable after step 3	**0.000**

Formula from the FP-300.6 end-to-end test: payable = 10000 − fee − (refund − fee reversed). The fee depends on the shop's MDR; reversal never exceeds the fee (BUG-056).

If the live demo fails: run the same flow in the test `apps/api/src/refunds/refundEndToEnd.test.ts` (FP-300.6) — it checks ledger, webhook, batch, timeline, reports and reconciliation for one refund.

UNDER LOAD

~21 paid checkouts/s from one IP, nothing lost under 3× overload



	VALUE
Estimate	**15–20 buyers at once, ≈ 21 paid/s, p95 ≈ 1.05–1.3 s**
Measured (k6, one IP)	15 users: 21.1 paid/s, p95 1.05 s, 0 wrong · 20 users: 22.5/s, p95 1.29 s
Floor	Acquirer simulator alone ≈ 0.55 s per checkout
Bottleneck	Postgres CPU (0.30 of 0.35, throttled 85%)
Overload 3×	2,637 shed with 503 in **~2 ms**; 927 paid
api killed (SIGKILL)	back in **2.1 s**; 927 paid = 927 captured; **0 lost, 0 doubled**
Slow-client IP	others: 0 paid → **231 paid**

HOW WE USED AI

Setup: Claude Code with our own `CLAUDE.md`, a push-deny permission file, read-only investigator + reviewer sub-agents, and a guard test that keeps the poisoned rules out.

AI WAS WRONG

Our own AI-written `CLAUDE.md` re-introduced a poisoned rule

Asked to replace the poisoned agent files, the AI said "done". Its new file quoted the old rule word for word: *"Math.random is fine for OTPs"*.

How we caught it

We wrote a guard test and ran it on **both** files. Originals: 0/7 (as expected). Ours: **6/7 — fail**:

```
CLAUDE.md still tells agents: Math.random for OTPs
```

Reworded, fixed the pattern → 7/7. The guard runs in every test run.

Also caught: "Claude says the check is redundant" → login without OTP (BUG-001) · AI estimated 4 GB cache, measured 467 MB · "380 tests" was 376. **The AI proposes; the code and the measurements decide.** Team sync: both our AI sessions read the same corrected `CLAUDE.md` + TEAM / PLAN / PROGRESS files, so Ebrahim's and Sayed's agents worked from one rulebook.

DECISIONS AND RISKS

What we chose not to change or run, and why

ITEM	WHY	SAFEGUARD WE PROPOSE
No rewrite of past money rows (CSV, scheme, UTC days)	Already paid or sent to the scheme: finance decision	Reviewed correction journal, not UPDATE
Checkout has no per-IP rate	So one-IP load meets the real valve, not a 429	Card-testing limits per link / card in the api
Crash between capture and finalize	Rare; 0 in the crash drill	Finalize in the capture transaction
Leaked acquirer secret in reference history	Not ours to rotate	Rotate; we refuse it at boot
Botnet floods	Per-IP limits can't stop many IPs	Cloud network protection
BUG-037 signup email reuse	Safe after BUG-002; shared emails are legitimate	Email verification (ENH-008)

APPENDIX · NOT PRESENTED

Test output, load-test numbers, findings by area

Tests (latest verified)

SUITE	BASELINE	NOW
API	353 / 365	**651/651**
Reporting	7 / 11	**15 / 15**
Go settlement	ok (no bug tests)	**all ok**
Web / admin	8 tests	44 tests + build / build

k6 steps (one IP, 45 s each)

USERS	PAID/S	P95	SHED 503	WRONG
10	14.6	1.04 s	0	0
15	21.1	1.05 s	0	0
20	22.5	1.29 s	0	0
25	18.5	1.83 s	15,553	0
30	18.7	1.28 s	20,747	0 real

Findings by area

AREA	CRIT	HIGH	MED
Login and access	5	5	1
Money coming in	6	0	0
Money maths	2	6	0
Settlement (Go)	3	3	1
Webhooks and reports	1	4	4
Refunds (FP-300)	3	2	1
Deployment	3	3	1
Poisoned AI files	1	0	0
Total 55	**24**	**23**	**8**

+ SEC-1…5, SEC-RT-1…6 (red team: 6 holes found and fixed, 77 attacks blocked), CAP-2/3. Flag: `FLAG{order_of_the_prompt_never_read_the_diff}`.

Reproduce it

Redeploy (one command)

`sh ops/prod/autodeploy.sh --force` — build at nice 19, migrate, `up -d`, record the commit when the api is healthy. Also runs by itself every 2 min from the fork.

Load test

```
psql … -v run=f -v count=60000 < ops/loadtest/make-links.sql
BASE=http://localhost:18080 sh ops/loadtest/steps.sh f 45s 10 15
20
k6 run -e PROFILE=overload -e RATE=60 -e DURATION=60s
checkout.js
psql … -v run=f < ops/loadtest/verify.sql
```

Crash drill result (verify.sql)

Paid links	927
Captured transactions	927
Paid without transaction	0
Wrong amount	0
Captured but not on a link	0
Reserved, no transaction (payable after 60 s)	15

Where to look

`docs/FINDINGS-REPORT.md` · `docs/bugs/` · `docs/challenge/CAPACITY.md` · `AI-USAGE.md` · `TRAPS.md` · `IMPLEMENTATION.md`